

Automated refinement of models for model-based testing using exploratory testing

Ceren Şahin Gebizli¹ · Hasan Sözer²

Published online: 14 September 2016
© Springer Science+Business Media New York 2016

Abstract Model-based testing relies on models of the system under test to automatically generate test cases. Consequently, the effectiveness of the generated test cases depends on models. In general, these models are created manually, and as such, they are subject to errors like omission of certain system usage behavior. Such omitted behaviors are also omitted by the generated test cases. In practice, these faults are usually detected with exploratory testing. However, exploratory testing mainly relies on the knowledge and manual activities of experienced test engineers. In this paper, we introduce an approach and a toolset, ARME, for automatically refining system models based on recorded testing activities of these engineers. ARME compares the recorded execution traces with respect to the possible execution paths in test models. Then, these models are automatically refined to incorporate any omitted system behavior and update model parameters to focus on the mostly executed scenarios. The refined models can be used for generating more effective test cases. We applied our approach in the context of 3 industrial case studies to improve the models for model-based testing of a digital TV system. In all of these case studies, several critical faults were detected after generating test cases based on the refined models. These faults were not detected by the initial set of test cases. They were also missed during the exploratory testing activities.

Keywords Model-based testing · Exploratory testing · Automated model refinement · Software test automation · Industrial case study

✉ Hasan Sözer
hasan.sozer@ozyegin.edu.tr
Ceren Şahin Gebizli
ceren.sahin@vestel.com.tr

¹ Vestel Electronics, Manisa, Turkey

² School of Engineering, Ozyegin University, Nişantepe Mah. Orman Sk. No: 34-36, Alemdağ – Çekmeköy, 34794 Istanbul, Turkey

1 Introduction

Model-based testing (MBT) is a testing technique that relies on models of the behavior of the system and/or its environment (Utting et al. 2012; Pretschner 2005; Utting and Legeard 2007). This technique involves the use of methods and tools for the automatic generation of test cases from these models (Utting et al. 2012). As such, the effectiveness of the generated test cases depends on the correctness and completeness of the employed models. In general, test models are created manually based on functional requirements specifications. Therefore, these models are subject to accidental omissions of certain usage scenarios and related system behavior. As a result, critical faults can be left undetected by the generated test cases, which do not cover the omitted system behavior.

Exploratory testing (ET) is commonly applied in practice as a complementary approach. In a typical ET process, test engineers make use of their domain knowledge and experience to perform tests. They continuously improve their testing strategies by using the new knowledge gained throughout the process (Tinkham and Kaner 2003a). ET has shown to be highly effective to detect critical faults (Itkonen 2011) and improve the overall effectiveness of the testing process (Itkonen et al. 2007). In practice, however, ET activities are by and large performed manually. The process relies on the domain knowledge and experience of test engineers. The gained knowledge and experience are not exploited effectively for supporting other techniques such as MBT.

In this paper, we propose an approach and a toolset, ARME (automated refinement of models for MBT using exploratory testing), for automatically refining test models based on the experience and domain knowledge gained during ET. ARME analyzes the execution traces that are recorded during ET activities. These traces are compared with respect to the possible execution paths in test models to identify omissions. Then, these models are automatically extended to incorporate any omitted system behavior. The models are used for (re)generating test cases that cover the extended parts of these models as well. The recorded traces are also used for updating model parameters. Test models are represented as Markov chains (Whittaker and Thomason 1994), in which transition probabilities are defined for switching among system states. These probability values are updated according to the frequency of visits to system states during ET activities. As a result, the generated test cases focus on the most frequently visited system states. Model refinement continues in an iterative manner as long as the ET activities continue.

We applied our approach in the context of 3 industrial case studies to improve the models for MBT of a digital TV system. In our first case study, we employed existing test models for testing one of the TV features. Three new faults were detected after we used ARME to refine these models. These faults were highly critical: one of them caused the TV to reset itself; another one was causing the installation of duplicate channels; and the activation of the third fault resulted in a crash of the system. Effectiveness of the generated test cases was further improved as a result of successive iterations of model refinement performed by ARME. In total, 5 new faults were detected after 4 iterations. Likewise, 7 and 6 new faults were detected in the second and the third case studies, respectively.

The rest of this paper is organized as follows. In the following section, we provide background on ET, MBT and our modeling approach. We present the overall process, methods and the ARME toolset in Sect. 3. The evaluation of the approach is discussed in Sect. 4 in the context of 3 industrial case studies. Section 5 summarizes the related studies. Finally, in Sect. 6, we conclude the paper and discuss future work directions.

2 Background

In this section, we provide background information regarding ET and MBT.

2.1 Exploratory testing

ET (Myers and Sandler 2004; Hetzel and Hetzel 1991) is characterized by a continuous learning and adaptation process, where the tester iteratively learns about the product and its faults, plans the testing activities, designs and executes the tests, and reports the results (Whittaker 2009). As the main difference from traditional software testing, ET is not based on a set of predefined test cases. Instead, testers use their creativity and experiences to steer the process dynamically. Test design, execution and learning are all concurrent activities in ET, which aims at using human effort efficiently by utilizing human intuition and experience such as domain knowledge, system knowledge or software engineering knowledge. There exist various definitions of ET (Bach 2003; Kaner 2006; Tinkham and Kaner 2003a). Bach briefly defines ET as *simultaneous learning, test design and test execution* (Bach 2003).

There are no formal descriptions or detailed methodologies defined for ET yet nor there exist strictly described procedures that should be followed by testers during test execution. That is why, ET has been mainly considered to be an *ad hoc approach*. Nevertheless, it is one of the mostly applied and one of the most successful approaches (Itkonen et al. 2007), also based on our observations in the industry.¹ ET activities are performed manually, and their success depends on the experience and skills of test engineers. Therefore, observations regarding ET cannot be generalized and ET cannot be considered as an alternative for formal and automated techniques such as MBT. However, it was previously showed that ET can be highly effective in practice to detect critical faults (Itkonen 2011). Hence, it can be considered as a promising complementary approach for other testing techniques. This fact also provided the motivation for the problem addressed in our work.

2.2 Model-based testing

Model-based testing (MBT) relies on test models to automatically generate test cases (Utting et al. 2012; Pretschner 2005; Utting and Legeard 2007). It has been employed in the industry for more than a decade to increase the efficiency of the testing process and for improving the software quality (Dustin et al. 1999). It relies on a model of the system under test (SUT), and/or its environment that is created mainly by analyzing system requirements. This test model defines the expected behavior of the SUT with respect to a set of inputs and actions of the user. The test model is provided as an input to a MBT tool, which automatically generates a set of test cases by traversing the possible behavioral scenarios on the model. These test cases are executed and compared with respect to the expected results to report any deviation from the expected behavior. In principle, the execution of test cases and the comparison of results can also be automated. However, MBT is mainly concerned with the automation of test case generation. This automation decreases the testing time and helps to achieve increased (and measured) coverage of possible execution scenarios. In addition, test models and generated test cases help to document and analyze the system behavior. Changes in requirements can be reflected to models for generating new test cases. Hence, this approach also reduces the maintenance effort.

There are several types of formalisms that are used for expressing the SUT model. These include UML models (Briand and Labiche 2002), finite state automata (FSA) (Chow 1978),

¹ We discuss our observations based on the industrial case studies in Sect. 4.3.4.

event sequence graphs (ESG) (Belli et al. 2014), state charts (Harel 1987), Markov chains (Whittaker and Thomason 1994) and labeled transition systems (LTS) (Tretmans 2011).

The use of UML models for MBT focuses in particular on the utilization of use case diagrams, sequence diagrams, collaboration diagrams and class diagrams (Briand and Labiche 2002). Hereby, test scenarios are mainly specified by sequence and collaboration diagrams. Use case diagrams are used for organizing these scenarios. There are one or more sequence or collaboration diagrams for each use case. Class diagrams are used for documenting the set of classes, methods and attributes that are involved in the scenarios.

FSA is a commonly utilized formalism for MBT, especially to represent state-based behaviors of a system. Informally, it involves a set of inputs, a set of states and a transition function that maps pairs of inputs and states to next states. A FSA model is scanned by a MBT tool for executable paths to generate test cases. Each possible sequence of states defines an execution path that can be specified as a test case. There exist several methods for generating such sequences including the transition tour (T) method (Naito and Tsunoyama 1981), the distinguishing sequence (D) method (Gonenc 1970) and the characterizing set (W) method (Chow 1978). In this work, we did not focus on test case generation; we focus on the refinement of test models instead. Test case generation is performed by the employed MBT tool using two different algorithms. One of these algorithms generates test cases in a stochastic manner. The other one satisfies full transition coverage.

ESG (Belli 2001; Belli et al. 2014) constitutes a more abstract representation compared to FSA. Hereby, inputs and states are represented together as *events*, each of which corresponds to a user-observable action. Informally, ESG involves a set of events and a transition function that maps events to next events.

State charts (Harel 1987) are similar to FSA models, but they can be hierarchically composed. In addition, states and transitions can be accompanied with boolean guards and a set of actions.

A Markov chain is also basically a FSA, in which probabilities are defined for state transitions (Whittaker and Thomason 1994). The system may change its state from the current state to another state or remain in the same state, according to a probability distribution. In our approach, we employ a form of extended Markov chains to represent test models (see Sect. 3.1). We exploit state transition probabilities to focus the scope of the generated test cases.

LTS models also comprise a set of states and transitions between different pairs of these states. In addition, they include a countable set of input labels and a countable set of output labels (Tretmans 2011). These two sets are disjoint. Each transition can be annotated with input and output labels. These labels are used for synchronization (Tretmans 2011) and modeling the input/output behavior of reactive systems.

Regardless of the employed formalism, test models are by and large created manually based on informal specifications. As a result, models can be incomplete and/or incorrect with respect to the actually implemented system. In the following section, we describe our approach for automatically refining test models by receiving feedback from ET.

3 Refinement of test models with ARME

The overall approach is depicted in Fig. 1. It consists of an iteration cycle among 3 basic processes: *Exploratory testing*, *model refinement* and *model-based testing*.

The iteration starts with an initial model developed for MBT. This model is created based on requirements specifications, independent of the ET activities. MBT is followed by

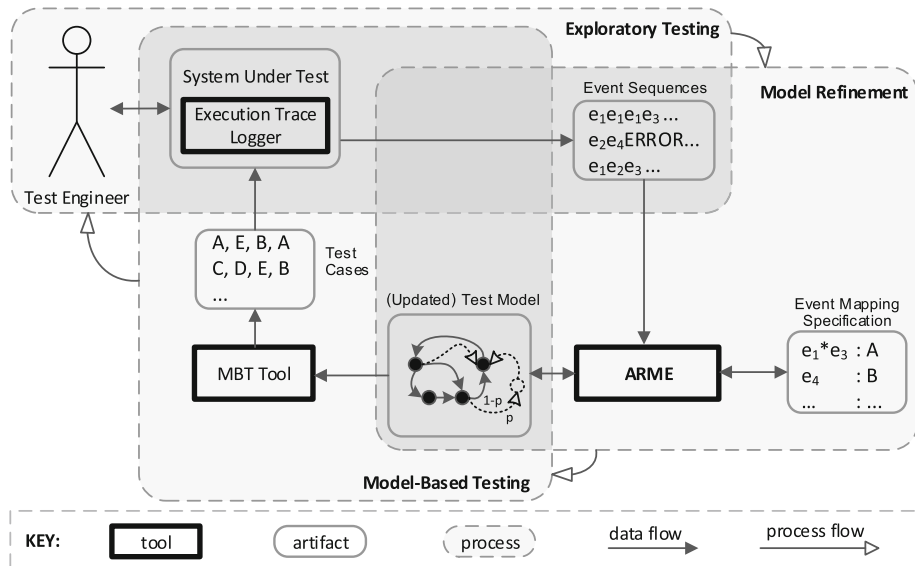


Fig. 1 Overall approach

ET. We do not interfere with the ET activities, where a test engineer interacts with the system. A built-in tool logs the sequences of events within the SUT throughout ET.

The collected *event sequences* are provided as input for the *model refinement* process, which is performed by the **ARME** toolset. **ARME** updates the existing test model that is used for MBT in the previous iteration cycle. The model involves states and transitions that represent possible user actions. On the other hand, execution traces are collected as a sequence of low-level events (function calls) that take place in the SUT. Hence, **ARME** takes a third input, namely the *event mapping specification*, which defines a mapping between the states in the model and the events taking part in the collected traces. This mapping is specified in the form of regular expressions. **ARME** analyzes the execution traces based on the provided mapping and compares them with respect to the possible execution paths in the model of the SUT. Then, the test model is automatically extended to incorporate any omitted system behavior. In the mean time, state transition probabilities are also updated according to the frequency of visited states calculated based on the collected event sequences.

After *model refinement*, the *model-based testing* process is repeated. The updated test model is provided to the MBT tool for re-generating test cases. These test cases are executed on the SUT. This iteration cycle can be repeated as long as ET activities continue and new execution traces are obtained. Hence, the number of cycles depends on the reserved test budget and available resources.

In the following, we explain our modeling and event mapping approach first. Then, we describe the two types of refinements performed by **ARME**: (1) adding missing states and transitions to test models and (2) updating state transition probability values.

3.1 Modeling approach and event mapping

In this work, we are focusing on event-based, reactive systems in particular. This is mostly the case with MBT approaches in general (Utting et al. 2012). In our case, we totally

ignore input dependent faults; inputs are basically abstracted away as a sequence of remote controller key press events for a TV system. In principle, our overall approach is agnostic to the type of tool and formalism that is used for creating test models for such a system. In fact, we previously applied our approach by using ESG models as well (Gebizli and Sozer 2014). However, the application of the approach was manual. In this work, we used MaTeLo² as the MBT tool. The ARME tool set is compatible with this tool for automated model refinement. The first reason for employing MaTeLo is to comply with the tools used for industrial case studies. The second reason is that this tool supports the use of Markov chain formalism. Markov chains provided us the ability to perform test case generation in a random-stochastic manner by utilizing transition probabilities. Otherwise, there is no particular reason for choosing this formalism or MaTeLo as the MBT tool. These were already employed by the software test department within the company. All the models that we used for case studies were also already developed with MaTeLo.

In fact, we do not use standard Markov chains but the so-called Extended Markov chains (Guen et al. 2004) that are developed with and utilized by the MaTeLo tool. In this formalism, transitions can be labeled with input and output vectors. Hereby, inputs and outputs are observable actions exchanged with the system and they are described with typed variables. Multiple input vectors can be associated with a transition. Each of these input vectors is associated with a probability value such that the sum of these probability values is equal to the transition probability. As such, this modeling formalism includes two levels of probabilities: (1) the probability to select the transition and (2) the probability to select an input vector given that the transition occurs (Guen et al. 2004). In our modeling approach, we defined only a single input vector for each transition. This vector includes the sequence of events (i.e., remote controller key presses) necessary to perform the corresponding transition. The sum of all the probability values for all outgoing transitions of a state has to be equal to 1. If there is only one outgoing transition for a state, then the probability value for that transition is 1 by default.

A sample test model is depicted in Fig. 2. The legend provided for this figure is also valid for other test models presented in the rest of the paper. Hereby, we can distinguish a start state and a terminal state on the top and the bottom of the figure, respectively. Test models have an hierarchical structure, and this one is a top-level model. States *Channel List*, *Installation*, *Browser* and *Record* include sub-models corresponding to these states. Each sub-model includes a set of states and transitions just like the top-level model. There can also be other sub-sub-models under the sub-models depending on the depth of the hierarchy.

During the ET activities, *Execution Trace Logger* collects execution traces in the form of sequences of function calls. For a digital TV, for instance, these function calls usually correspond to the calls that are triggered when the user presses a key on the remote controller. On the other hand, test models include states and transitions that are represented at a higher level of abstraction. Sequences of events trigger transitions, and they are specified as inputs for the corresponding transitions. In our approach, a sequence of events that are specified as input for a transition is mapped to the target state of that transition. As such, an *event mapping specification* is created manually to map a sequence of low-level events to states in the model. The relevant sequences of low-level events are defined in the form of regular expressions.

² <http://www.all4tec.net>.

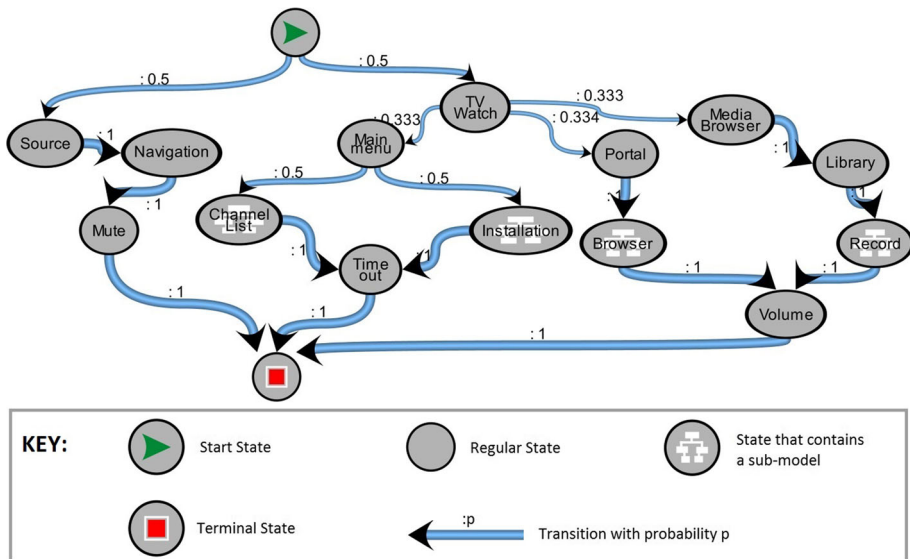


Fig. 2 Sample top-level model

There are 3 basic assumptions regarding the *event mapping specification*. First, we assume that this specification is correct and complete. It maps each state of the model to a sequence of events (function calls).

Second, we assume that sequences of events that map to states are disjoint. That is, a sequence that is mapped to a state cannot be a sub-sequence taking place in another mapping. In fact, all sequences start with a different event in our case. This makes it scalable to check if a sequence matches to a mapping definition.

Third, we assume that a whole sequence of events that cannot be mapped refers to a new (omitted) state. It might be the case that such sequences represent more than one state. However, it is not possible to decompose such unrecognized sequences of events to multiple states without the domain knowledge. A domain expert can possibly control new states added by the tool and modify/decompose them manually, if deemed necessary.

3.2 Extending test models

In this sub-section, we describe the model extension procedure in detail. The procedure is outlined in Algorithm 1, which takes different cases into account. We explain the conditions and refinement operations for each of these cases. We also illustrate these cases based on the sample model that was provided in Fig. 2. Note that Algorithm 1 refers to a single model refinement step that is iteratively executed as part of the overall approach.

As a running example for describing the model refinement on sample model in Fig. 2, consider an event mapping specification listed in Listing 1. Assume that a sequence of events is recorded during ET as: “a, b, x, y, e, f, c, d, ..., hl, i, ..., kl, lm, lm, m, n, pr, ...” where “x” and “y” are not defined in the specification.

The refined model with sample extensions regarding the different cases is depicted in Fig. 3. We refer to the corresponding parts of this model, while explaining different cases considered by the algorithm.

Listing 1 An example event mapping specification.

```

1  // <Event Sequence>: <States>
2      <a><b>          : Navigation;
3      <c><d>          : Mute;
4      <e><f>          : Source;
5      ...
6      <h><l>          : Playing Record under "Record";
7      <i>             : Youtube under "Browser";
8      ...
9      <kl><lm>        : Antenna Settings under "Installation";
10     <n><pr>         : Advanced Channel List under "Ch. List"
11     ...
12  //

```

Algorithm 1 Model Extension Procedure

```

1: Input: event sequence, test models
2: procedure EXTENDMODEL
3:    $S_P \leftarrow \text{start-state}$ 
4:   while sequence is not consumed do
5:      $S_F \leftarrow \text{get next mapped state}$ 
6:     if  $S_F$  is not defined then
7:       create event sequence  $E$ 
8:       while  $S_F$  is not defined do
9:          $e \leftarrow \text{skipped event from sequence}$ 
10:        append  $e$  to  $E$ 
11:         $S_F \leftarrow \text{get next mapped state}$ 
12:      end while
13:      create new state  $S_N$ 
14:      define mapping  $S_N : E$ 
15:      define transitions  $S_P$  to  $S_N$  and  $S_N$  to  $S_F$ 
16:    else
17:      if transition is not defined from  $S_P$  to  $S_F$  then
18:        if  $S_P$  and  $S_F$  are in the same model then
19:           $S_1 \leftarrow S_P$ 
20:           $S_2 \leftarrow S_F$ 
21:        else
22:           $S_1 \leftarrow \text{upper level state of } S_P$ 
23:           $S_2 \leftarrow \text{upper level state of } S_F$ 
24:        end if
25:        if direct transition between  $S_1$  and  $S_2$  is not allowed then
26:           $S_{Pre} \leftarrow \text{common predecessor state for } S_1 \text{ and } S_2$ 
27:           $S_{Suc} \leftarrow \text{common successor state for } S_1 \text{ and } S_2$ 
28:          define transition  $S_{Suc}$  to  $S_{Pre}$ 
29:        else
30:          define transition  $S_1$  to  $S_2$ 
31:        end if
32:      end if
33:    end if
34:     $S_P \leftarrow S_F$ 
35:  end while
36: end procedure

```

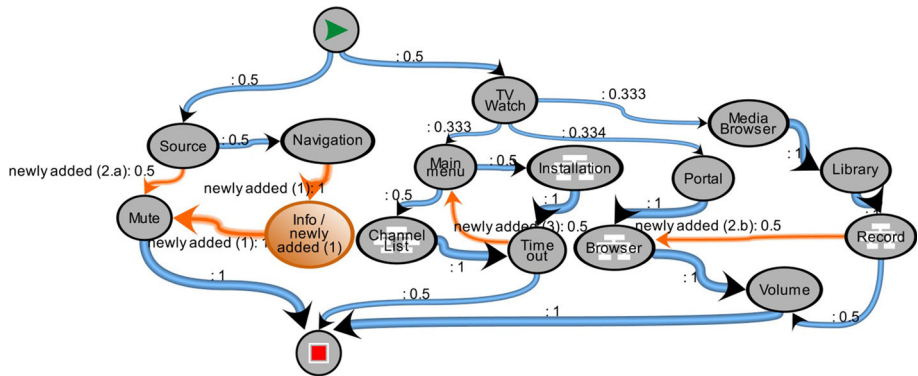


Fig. 3 Sample model view after refinements

The procedure (Algorithm 1) has one main loop (Lines 4–35). In this loop, the recorded event sequence is consumed until the end and transitions from one state (S_P) to another (S_F) is traced on the model one at a time. S_P is initialized as the *start* state before the loop (Line 3). In each step, the next sequence of events is mapped to S_F (Line 5). If such a state can be mapped, and if there is a transition defined between S_P and S_F , then S_F becomes the new S_P (Line 34) and the process continues with the next sequence of events. A model extension is considered for the following cases.

1. If an event sequence cannot be mapped to a state after S_P , a state is deemed missing in the model (Lines 6–16). This state should take place between S_P and the next state S_F that can be mapped in the event sequence. In this case, a new event sequence, E , is created first (Line 7). Events are skipped one by one (Line 9) as long as the remaining sequence cannot be mapped to a state (Line 8). In the mean time, E is appended with all the skipped events (Line 10). Once S_F is found, a new state, S_N is defined and a new mapping is added for this state to be associated with the event sequence E (Line 14). Then, two new transitions are defined from S_P to S_N and from S_N to S_F (Line 15). Based on the running example, ARME recognizes that “x” and “y” are not defined, so it builds a regular expression from these traces and maps them to a state called “newly added (1)” and connects the transitions to this new state to complete the model. This case is illustrated with the set of extensions marked with orange color and labeled as *newly added (1)* in Fig. 3.
2. If S_F is mapped (Line 16) but there exists no transition from S_P to S_F (Line 17), then a transition is missing. We consider two sub-cases for this case:
 - (a) If S_P and S_F are part of the same model, then we can directly add a transition from S_P to S_F . As such, they are recorded as S_1 and S_2 , respectively (Lines 19–20).
Based on the running example, there is a path from states *Source* to *Mute*, so ARME adds a new transition [newly added (2.a)] between these states. This case is illustrated with the extension marked with orange color and labeled as *newly added (2.a)* in Fig. 3.
 - (b) If not, we find their *upper-level* states. A state S_U is an upper-level state of S if S_U is associated with a sub-model and S takes place in that sub-model. If a state S takes place in the top-level model, then the upper-level state of S is itself.

Let's say the top-level model includes states A and B such that S_P takes place in the sub-model of A and S_F takes place in the sub-model of B . We cannot directly add a transition from S_P to S_F in this case. They reside in two different sub-models. Instead, we can add a transition from state A to B , which are defined as part of the same model (i.e., the top-level model). As such, states A and B are recorded as S_1 and S_2 , respectively (Lines 22–23).

Based on the running example, ARME recognizes the execution traces “..., hl, i, ...” which are mapped to two states: *Playing Record* and *Youtube*. Since these states are not part of the models at the same level, it cannot directly add a transition between them. Therefore, it finds their upper-level states: *Record* is upper-level state of *Playing Record* and *Browser* is upper-level state of *Youtube*. Then it adds a new transition [newly added (2.b)] from *Record* to *Browser*. This case is illustrated with the extension marked with orange color and labeled as *newly added (2.b)* in Fig. 3.

3. It might not be possible to make a direct transition from S_1 and S_2 for the 2.b case above. In this case (Line 25), we create a cycle by adding a transition from the common successor back to the common predecessor of these states (Line 28). Then, it becomes possible to first visit S_1 , then follow the cycle to visit the common predecessor of S_1 and S_2 again, and then visit S_2 .

Based on the running example, ARME recognizes execution traces “..., kl, lm, lm, m, n, pr, ...”, which are mapped to two states: *Antenna Settings* and *Advanced Channel List*. These states are not at the same level. Therefore, it finds their upper-level states. A direct transition between these states are not allowed. So, ARME adds a new transition (newly added (3)) from the common successor back to the common predecessor. This case is illustrated with the extension marked with orange color and labeled as *newly added (3)* in Fig. 3. We directly add a transition from S_1 and S_2 as determined in the second case (2.a or 2.b) if this constraint does not apply (Line 30).

Items 1 and 2 above are generic, whereas item 3 is related to a (possible) constraint regarding the application domain. A direct transition might not be allowed among a group of states due to a domain-specific or application-specific constraint. In our case studies, for instance, we employed test models that are designed for digital TV systems. These models are designed such that direct transitions among the sub-menu items are not possible. The user has to go back to the main menu first. This can happen automatically after a timeout period; as such, this change might not be visible in the sequence of user events. There exists a timeout state for each menu group, and one should go through this state and the state that represents the main menu before switching to another menu. If such constraints exist, one must explicitly specify (tag) groups of states, among which there can be no direct transitions. In our case, we can automatically identify and tag states that represent TV sub-menu items since they follow a pattern: a group of states represent sub-menu items if these states have in-degree and out-degree values equal to 1, if they have the same predecessor and if they have the same successor, which is not the terminal node. In the running example, for instance, the upper-level states of *Antenna Settings* and *Advanced Channel List* have in-degree and out-degree values equal to 1. They have also the same predecessor and the same successor, which is not the terminal state.

In the following sub-section, we describe refinements that are performed regarding state transition probability values in the model.

3.3 Updating state transition probability values

In our approach, we not only extend models for missing paths, but we also refine probability values according to the usage profiles collected during ET. The numbers of visits to different system states are counted for subsequent iterations of ET activities. For each iteration i , a probability value, $P_{\text{calc}}(t)^i$ is calculated for each transition t , according to Eq. 1.

$$P_{\text{calc}}(t)^i = \frac{\# \text{ of visits for target } (t)}{\sum_{\forall t', \text{source}(t)=\text{source}(t')} \# \text{ of visits for target } (t')} \quad (1)$$

Hereby, the number of visits to the target state of the transition is calculated as the nominator. The denominator is calculated as the sum of the numbers of visits for all the alternative transitions. These are the transitions, which have the same source states as t . The calculated $P_{\text{calc}}(t)^i$ values in each iteration are not directly assigned as the probability values on transitions. Instead, they are reflected to the calculations in subsequent iterations according to Eq. 2.

$$P(t)^i = \begin{cases} \frac{1}{\# \text{ of alternative transitions}} & i = 0 \\ P(t)^{i-1} - \frac{P(t)^{i-1} - P_{\text{calc}}(t)^{i-1}}{k} & i \geq 1 \end{cases} \quad (2)$$

Initially, default probability values are assigned for the first iteration. Let's assume for a state s that there exist n outgoing transitions to n different states. Then, for each of these transitions, the probability value is calculated as $P(t) = 1/n$ when $i = 0$. In subsequent iterations, this value is updated according to the $P_{\text{calc}}(t)$ calculated in the previous iteration. Hereby, k is a coefficient ($k > 0$) that allows us to adjust the weight of the $P_{\text{calc}}(t)$ value on calculating $P(t)$. If $k = 1$, the calculation of $P(t)$ does not depend on the previous $P(t)$ value but is completely determined by $P_{\text{calc}}(t)$ that is calculated in the previous iteration. Previously calculated $P(t)$ value has more influence on the calculation as k increases.

In our case studies, we fixed the value of k as 2. In the following section, we explain these studies and discuss the results.

4 Industrial case study: digital TVs

In this section, we introduce 3 industrial case studies for the application of our approach to digital TV systems. Our subject systems are developed by Vestel,³ which is one of the largest TV manufacturers in Europe. There are hundreds of test suites being used for testing these systems. These test suites are used for testing different features of the system, and/or they are created for different types of tests such as performance tests, certification tests, connectivity tests and functional tests. Some of them are manually created. Some others are automatically generated by employing MBT. Many models have been created for this purpose. They are represented as extended Markov chains, some of which include thousands of states and transitions. MaTeLo⁴ is used for model creation and test case generation. All the test cases

³ www.vestel.com.tr.

⁴ <http://www.all4tec.net>.

are automatically executed with an in-house developed test automation tool. In parallel, manual exploratory tests are performed by experienced test engineers.

We performed case studies with 3 different models. The same software version was used throughout the studies, and no faults were fixed to keep the results consistent. The first model is related to the channel installation, named as Digital Video Broadcasting-Terrestrial Channel Installation (DVB-TCI). The second one is for testing the media browser (MB) module, which allows the users to open video, music, or picture files on TV via external devices. The third one is created for random torture test (RTT). RTT is actually *stress testing*, which aims at discovering how the system behaves under sustained use. The term *torture* is adopted within the company instead of stress testing, and it was reflected to the naming of test models. The RTT test model involves the use of several features that are expected to be mostly used by the end users.

4.1 Research questions

Our goal is to refine test models iteratively and to be able to generate more effective test cases/steps by using the refined models for detecting more faults. As such, we defined the following 4 research questions:

RQ1 To what extent models are refined by ARME in terms of updated or newly added states and transitions?

RQ2 To what extent successive iterations of the approach contribute to finding new faults?

RQ3 To what extent ARME contributes to finding new faults compared to MBT and ET?

RQ4 How does ARME affect test suite size in terms of the number of test steps?

In the following, we explain the experimental setup we used for evaluation. Then, we present and discuss the obtained results for addressing the research questions. We also illustrate steps of our approach using the case studies as a running example.

4.2 Experimental setup

Test models We used 3 different test models that were previously created for DVB-TCI, MB and RTT based on requirement specifications documents. All the 3 models were already being used in production. They were developed by the software test department within the company prior to our studies. We learned that the test models regarding DVB-TCI and MB were developed by manually analyzing the corresponding requirements specification documents. In addition, usage profiles were analyzed for the development of the RTT model. Mostly used features were identified from these profiles, and the model was developed to define the behavior just for these features. Figure 4 depicts the top-level RTT model.

Mapping specifications We created a mapping specification for each of the test models. As an example, Listing 2 shows a part of the mapping specification created for the MB model.⁵ Hereby, each state is represented by a regular expression. For instance, we can see at Line 5 of Listing 2 that a function call named *p*, followed by zero or more function calls named *v* maps to the state *Playing Video*. These expressions are defined manually to map a sequence of low-level events to states in the model. There is usually a specific function call that handles a remote controller command. For this reason, a single function call is mapped to a state in most cases.

⁵ Due to confidentiality, we do not disclose the real function names used in the implementation.

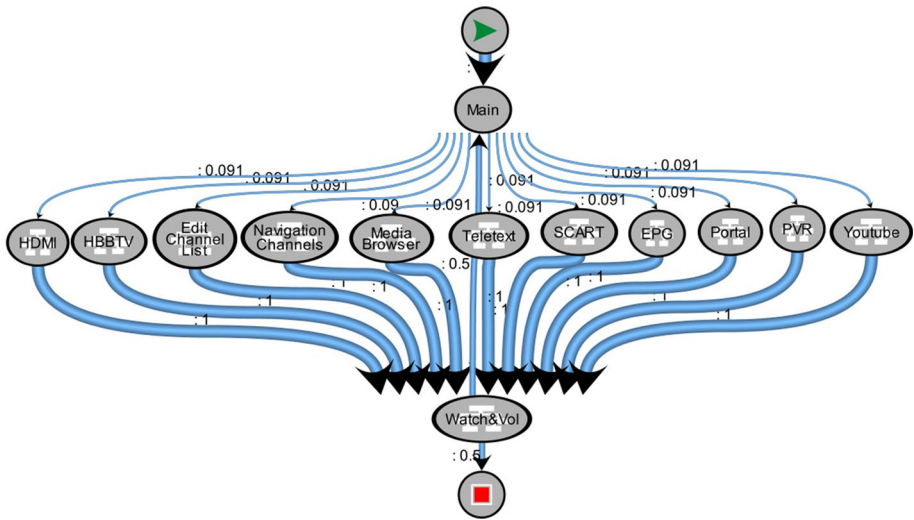


Fig. 4 Snapshot of the initial test model for RTT

Listing 2 Part of the event mapping specification for the MB model.

```

1  // <Event Sequence>                                : <States>
2      <mansr><f>*<snd>                                : Tune to DVB-S channel;
3      <mb><mn>                                          : MB menu;
4      <v><mn>                                           : Video option of MB menu;
5      <p><v>*                                           : Playing Video;
6      <sub>                                             : Press Subtitle;
7  //
```

Test case generation Based on the developed models, test cases are generated automatically with two different test case generation algorithms that are provided by the MaTeLo tool. For the first two case studies, test models were designed to cover all the features as documented in requirement specifications. We employed the so-called *Minimum Arc Coverage* algorithm (Joye 2014). This algorithm is also known as *Chinese Postman* (Joye 2014). We used this algorithm with default parameters, in which the coverage criterion is basically transition coverage. Test case generation stops when all the transitions in the model are visited at least once.

For the third case study, where updated transition probability values are utilized, test models were designed for stress testing. Hereby, the goal was not to cover all the features; on the contrary, testers were focusing on a set of specific features, while ignoring some other features. Hence, we exploited execution traces to obtain a usage profile and refine transition probabilities accordingly. We employed the so-called *User Oriented* algorithm (Joye 2014) to generate test cases based on these refinements. This algorithm has two parameters: (1) the maximum number of test steps within a test case and (2) the maximum number of test cases. We set the maximum number of test steps to 5000 (a limit that we never reached). We set the maximum number of test cases to 1. As a result, this algorithm starts from the start state and traverses the model by choosing a transition at each state non-

deterministically based on transition probabilities. It does not guarantee transition coverage. It generates test cases by using the probability values annotated on the transitions of the model. If a probability value of a transition which is going out of a state is higher than other transitions going out of the same state, then the algorithm will choose that transition stochastically. The algorithm terminates when the final state is reached. The generated test cases for all the 3 case studies are used in production.

ET activities ET activities are performed by 3 test engineers by focusing on the features in the scope of these case studies. The number of engineers and test durations were not deliberate choices made by us. ET activities were already being performed regularly in the company. We just collected data without interfering with the process. During the ET activities, all the execution traces are logged. We asked test engineers to save the records that were logged during tests. We also asked them to record each fault they found.

Faults found by test cases generated from the initial test models and faults discovered during the initial ET activities For the DVB-TCI case, with the initial model, 22 test cases were generated, which include 261 test steps in total. After executing the generated test cases, 3 faults were found. During the ET activities, in total 5 faults were found including the 3 faults which were found with the test cases from the initial model.

For the MB case, with the initial model, 107 test cases were generated, which include 809 test steps in total. After executing these test cases, 36 faults were found. During the ET activities, in total 44 faults were found including the 36 faults which were found with the test cases from the initial model.

For the RTT case, with the initial model, 1 test case was generated which includes 950 test steps in total. After executing the test case, 1 fault was found. During the ET activities, in total 2 faults were found including the 1 fault which was found with the test case from the initial model.

These results show that ET activities are highly effective in detecting faults. In all the case studies, ET found all the faults revealed by MBT. Besides, test engineers found some additional faults which had not been found with MBT before. These results support the motivation for our approach. In the following, we show that these results can be further improved by refining existing test models with ARME based on ET. We can detect further new faults, which were missed by both ET and MBT before refinements of ARME.

4.3 Results and discussions

In this section, we evaluate the results obtained for the 3 different case studies and discuss our experiences from the perspective of the 4 research questions.

4.3.1 Refinement of models by ARME (RQ1)

ARME has introduced many extensions to test models based on the collected execution traces during ET activities. For instance, Fig. 5 depicts the top-level DVB-TCI model previously created with the MaTeLo tool. The original model did not consider the fact that after going into one of the sub-menu items, the user can go back to the main menu directly. There was no such a transition defined. ARME added this missing transition to the model. The corresponding transition introduced by ARME is marked by painting it in orange color. For this model, there was a missing transition between the states of two different sub-models. These sub-models were representing sub-menu alternative options. Therefore, a direct transition between them was not allowed. As a result, a transition was added from the common successor state back to the common predecessor state in the top-level model.

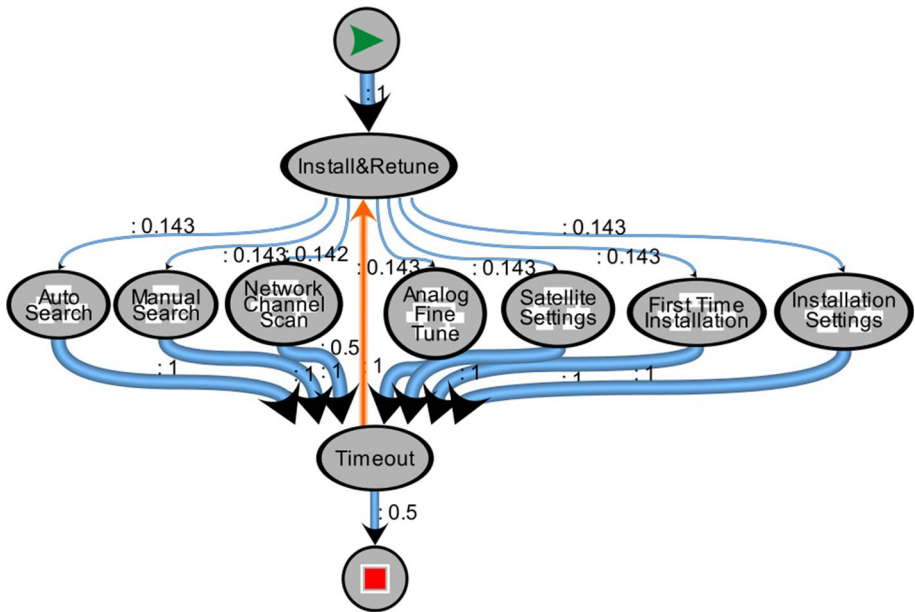


Fig. 5 Refined top-level DVB-TCI model

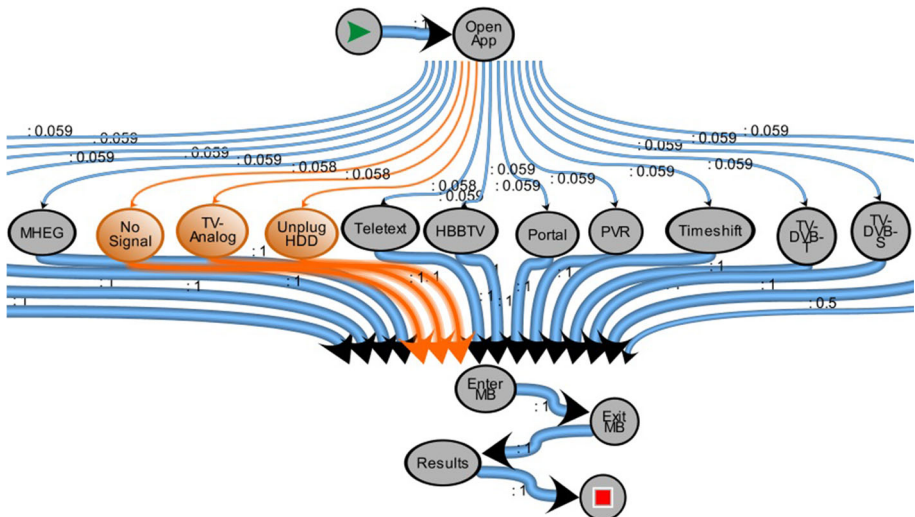


Fig. 6 Refined enter/exit MB top-level model

For the MB model, ARME added several new states and transitions. Some of the refined models are depicted in Figs. 6, 7 and 8, where modifications are marked with orange color. The first two rows of Table 1 show the overall updates performed on test models after all the refinement iterations for the DVB-TCI and MB case studies. For the DVB-TCI case study, the number of added states is 3 and the number of added transitions is 6. For the MB case study, the number of added states is 15 and the number of added transitions is 23. We

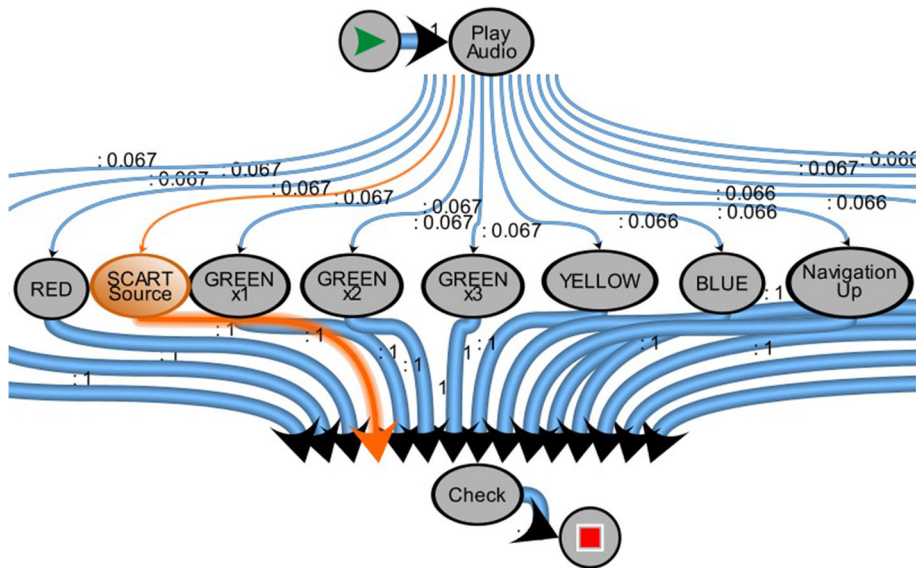


Fig. 7 Refined playing audio sub-model

did not update the transition probabilities for these two case studies. On the other hand, we can see that there are no new states or transitions added to the test models for the RTT case study. However, all the transition probabilities in the top-level model are updated.

Updates of probability values on state transitions are performed iteratively as explained in Sect. 3.3. For instance, Table 2 shows the updates performed on the top-level RTT model after the first iteration. Hereby, the first column lists the alternative state transitions from the *start* state. The second column shows the initial probability values before the update. They are all equal to each other by default. The third column lists the number of visits to the corresponding states during manual testing activities. The fourth column shows the calculated probability values (i.e., P_{calc}) according to the number of visits. Finally, the last column lists the updated probability values calculated based on Eq. 2. Hereby, the states that are listed in the first column are those states that are directly reachable from the start state in the top-level model. Hence, the updated probability values as listed in the last column are assigned to transitions, where the source of the transition is the start state and the target of the transition is the state that is listed in the first column of the same row. These values are re-calculated and replaced throughout iterations. The number of iteration cycles depends on the reserved test budget and available resources. In our case studies, we did not have any control over these. We repeated the cycle until new faults are not detected by model refinement. That is, we used the absence of new faults as the stopping criterion.

4.3.2 New faults detected by ARME in successive iterations of model refinement (RQ2)

To address RQ2, for each case study, we first recall the initial results that were obtained before any model refinement was performed. Then, we list the results obtained after each successive iteration of model refinement performed by ARME.

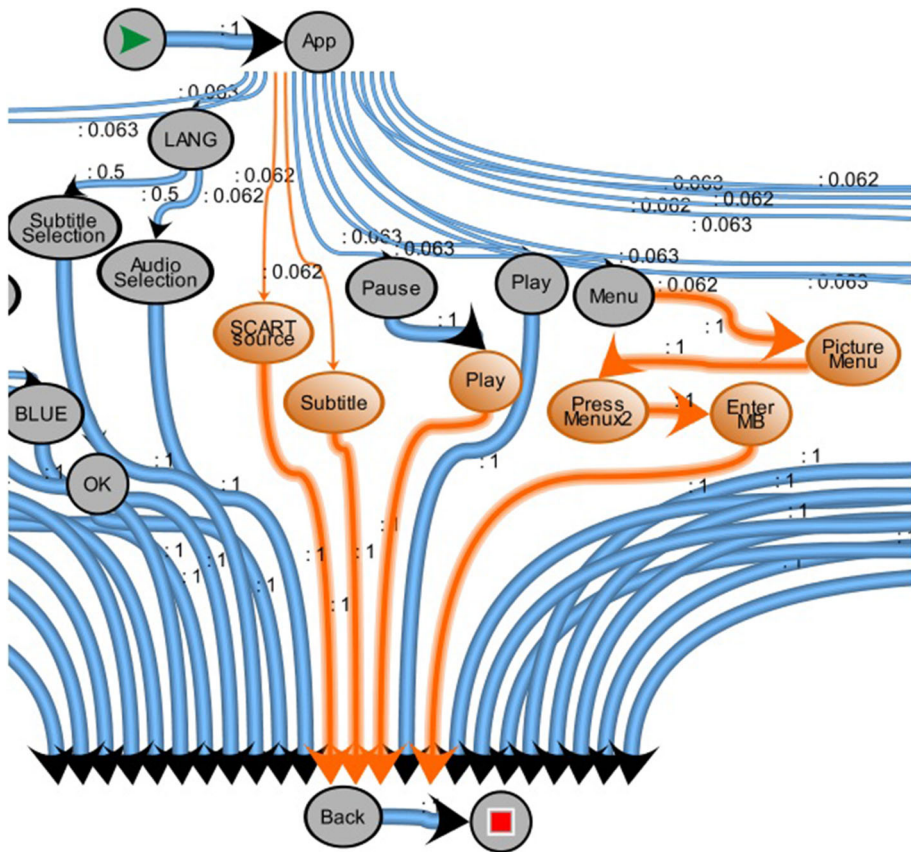


Fig. 8 Refined playing video sub-model

Table 1 Refinement of test models throughout iterations

Case study	# of added states	# of added trans.	# of updated trans.
1. DVB-TCI	3	6	0
2. MB	15	23	0
3. RTT	0	0	11

For the DVB-TCI case, 22 test cases were generated from the initial model which was created based on requirement specifications. After executing these test cases, 3 faults were found. Independent from this, ET activities were performed on DVB-TCI, and 5 faults were found. These 5 faults include the 3 faults which were found with the test cases generated from the initial model before. That is, 2 new faults were found with ET activities. Then, we refined the initial model by taking the execution traces from ET activities. We regenerated the test cases from the refined model, and we executed this new set of test cases. This time, 8 faults were found in total. 5 of them were found before. However, 3 new faults were found. These faults have not been found before either with MBT using the

Table 2 Calculated probability values for the RTT case study

State-module	Init. prob.	# of visits	Calculated prob.	Updated prob.
EPG	0.091	7	0.093	0.092
Portal	0.091	4	0.053	0.072
Youtube	0.091	12	0.16	0.125
HBBTV	0.091	0	0	0.045
Media Browser	0.091	3	0.04	0.066
PVR	0.091	6	0.08	0.086
HDMI	0.091	1	0.013	0.052
SCART	0.091	3	0.04	0.066
Teletext	0.091	2	0.026	0.059
Edit Channel List	0.091	3	0.04	0.066
Navigation	0.091	34	0.45	0.27

Table 3 Properties of the DVB-TCI model and the number of faults found throughout successive iterations of model refinement

Model	# of states	# of trans.	# of test cases	Total # of test steps	# of new faults detected by ET	# of new faults detected by ARME	Total # of faults
Initial	85	106	22	261	–	–	3
Iter. 1	86	109	1	252	2	3	8
Iter. 2	86	110	1	265	1	1	10
Iter. 3	88	112	1	267	0	1	11
Iter. 4	88	112	1	267	0	0	11

initial model or with ET activities. Moreover, these 3 faults were highly critical; one of them caused the TV to reset itself after a channel search was performed; another one was the reason for duplicate channels added at the end of channel search; the activation of the third fault resulted in a crash of the system. We continued to take execution traces from new ET activities that focused on the DVB-TCI feature. Each time, we refined test models with ARME based on new traces obtained. Then, we applied MBT based on the refined model. We performed these steps iteratively until there was no new fault detected. The results of the DVB-TCI case study are summarized in Table 3. The first row lists the initial results obtained with the original model. The following rows show the results after successive iterations. We can see that new faults were discovered after every iteration of model refinement. We can also notice that the number of test cases was decreased to 1 after the first iteration. This is due to the loop introduced by ARME (see Fig. 5), which makes it possible to cover many execution paths in a single test case.

Initially, 107 test cases were generated for the MB case, which include 809 test steps in total. After executing the generated test cases, 36 faults were found. However, 8 more faults were discovered during the ET activities. After the refinement of the test models based on these activities, 853 test steps were generated. Forty-eight faults were found when these test cases were executed. Thirty-six of these faults had been already detected before with the initial model. Twelve new faults were discovered, 8 of which had been discovered during the ET activities. Four of the new faults had not been detected before neither with the initial model nor with the ET activities. These 4 faults were highly critical as well; the

Table 4 Properties of the MB model and the number of faults found throughout successive iterations of model refinement

Model	# of states	# of trans.	# of test cases	Total # of test steps	# of new faults detected by ET	# of new faults detected by ARME	Total # of faults
Initial	171	298	107	809	–	–	36
Iter. 1	181	314	113	853	8	4	48
Iter. 2	183	316	113	861	1	2	51
Iter. 3	186	321	115	873	0	1	52
Iter. 4	186	321	115	873	0	0	52

first one made the audio/video output disappear; the second one corrupted remote controller key buffers; the third one made user commands undetectable; and the last one caused the TV to reset itself. These faults were detected after the first model refinement only. Overall results of this study are summarized in Table 4. The first row lists the initial results obtained with the original model. The remaining rows list the results obtained after successive iterations of model refinement. We can see that new faults were discovered after every iteration of model refinement. We can also observe that the number of test cases increases by 6 after the first iteration. However, 12 new faults were discovered. This is because, more than one fault can be found with one test case.

For the RTT case, initially, 1 test case which includes 950 test steps was generated. After executing the generated test case, 1 fault was found. However, 1 more fault was discovered during the ET activities. After the first model refinement based on these activities, 940 test steps were generated. Four faults were found when the new test case was executed. One of these faults had been already detected before with the initial model. Three new faults were discovered, one of which had been discovered during the ET activities. Two of the new faults had not been detected before either with the first model or with the ET activities. These 2 faults were highly critical as well: the first one caused a crash of an application and the second one corrupted remote controller key buffers after HBBTV⁶ operation. We can see these results in the second row of Table 5 (Iteration 1). In Table 5, we can also see the results obtained with the initial model in the first row (Initial) and results that are obtained after successive model refinements. These refinements are continued until the fifth iteration after which no new faults were detected. We can see that new faults were discovered after each of the first four iterations of model refinement. Note that the number of test steps decreases while the number of test cases remains same. At the same time, the number of detected faults increases. This is because, at each iteration, test case generation focuses on different execution paths according to the updated probability values on state transitions.

4.3.3 Overall contribution of ARME in finding new faults compared to MBT and ET (RQ3)

Results of the case studies show that we can identify critical faults by refining test models based on ET with the help of ARME. Our approach and the tool support enabled the transfer of knowledge from manual ET activities to MBT. Table 6 shows the comparison of

⁶ HBBTV (<http://www.hbbtv.org/>) stands for Hybrid Broadcast Broadband TV. It is an initiative for harmonizing the broadcast/broadband delivery of entertainment services for TVs and set-top boxes.

Table 5 Properties of the RTT model and the number of faults found throughout successive iterations of model refinement

Model	# of states	# of trans.	# of test cases	Total # of test steps	# of new faults detected by ET	# of new faults detected by ARME	Total # of faults
Initial	87	116	1	950	–	–	1
Iter. 1	87	116	1	940	1	2	4
Iter. 2	87	116	1	929	1	2	7
Iter. 3	87	116	1	923	0	1	8
Iter. 4	87	116	1	936	0	1	9
Iter. 5	87	116	1	927	0	0	9

Table 6 Comparison of the number of faults found with the initial test models, with ET activities and with ARME

Case study	# of found faults with initial MBT	# of Found new faults with ET	# of Found new faults with ARME	Ratio of new faults found by ARME (%)
1. DVB-TCI	3	3	5	45.45
2. MB	36	9	7	13.46
3. RTT	1	2	6	66.66

the number of faults found with the initial test models, with ET activities and with ARME after all the successive iterations for each of the 3 case studies.

For the DVB-TCI case study, 3 faults were found with the initial test model. Then, 3 additional faults were revealed during the ET activities. In addition to these 6 faults in total, 5 more new faults were detected with ARME. Hence, 45.45 % of the overall faults were detected only by ARME. Likewise, 7 and 6 new faults were detected only by ARME in the second and the third case studies, respectively. These faults represent 13.46 % of the overall faults for the second case study, whereas the ratio is 66.66 % for the third case study.

We observed that the effectiveness of the models used for MBT can be significantly increased by exploiting feedback from ET. Integration of the two testing techniques is highly effective in detecting faults and reusing domain knowledge. We showed that our approach helps to detect critical faults and increases the efficiency of the testing process. As an initial investment, the approach requires additional effort for defining the mappings of events to states. However, this is a one-time effort. Without ARME we would only detect the faults that are revealed with the initial MBT and the ET activities only. We could still refine test models manually, at least whenever a new fault is discovered during ET activities; however, for each such case, we would have to debug the model and the execution traces to reveal why the fault was not detected and we would have to manually identify missing edges and states.

4.3.4 Effects of ARME on test suite size (RQ4)

We compared the test suit sizes that are obtained with the initial test models and those that are obtained after successive iterations of model refinement with ARME. Table 7 shows the difference in terms of the total number of test steps. In the first case study for the DVB-

Table 7 Difference of test suite sizes in terms of the total number of test steps after successive iterations of model refinement with ARME

Case study	Total # of test steps with initial MBT	Total # of test steps after ARME	Difference in the size of test steps (%)
1. DVB-TCI	261	267	+2.3
2. MB	809	873	+7.91
3. RTT	950	927	−2.42

TCI model, the number of test steps was increased by 2.3 %. In the second case study for the MB model, the number of test steps was increased by 7.91 %. In the third case study for the RTT model, the number of test steps was decreased by 2.42 %. Results show that ARME does not always lead to significantly larger test suites compared to those obtained with the initial test models. On the contrary, refinements of ARME can even decrease the number of test steps although the execution of these steps enables the detection of new faults. Hence, test suites become more effective in terms of the number of detected new faults per executed test step.

4.4 Assumptions and limitations

We have two basic assumptions regarding our approach. First, we assume that the collected traces always reflect correct, possible user behavior. Second, we assume the availability of test engineers who regularly perform ET activities. The general applicability of our approach is constrained by our assumptions regarding the application domain and availability of experienced test engineers for performing ET activities in the working context.

The second assumption is a crucial one since the main purpose of our approach is to exploit feedback received from ET activities to refine testing models. Our approach is not usable without the availability of such feedback.

The first assumption limits the set of possible subject systems since they have to be event-based systems in which all user allowed actions are correct. That is, the system should not crash or lock due to an unexpected sequence of user actions. Although this assumption brings in a limitation, most of the consumer electronics products such as smart phones, refrigerators, washing machines, dishwashers, air conditioners and cookers can be considered as candidate subject systems that conform to our assumption. In fact, we performed another preliminary study for a camera application of smart phones produced by the same company. We detected a new fault after we refine its test model based on execution traces obtained from ET activities. We have not reported this study in detail since it was immature at the time of writing this paper.

Our model refinement algorithm considers multiple cases for deciding on how to extend the given test models (Recall Sect. 3.2). One of these cases considers an application-specific constraint, in which a direct transition is not allowed among a group of states in the test model. If such a constraint exists, one must explicitly specify (tag) the corresponding groups of states. In our case, we could automatically identify and tag such states since they conform to a pattern in the test models. This might not be possible for all types of systems. In principle, one can remove this case from the algorithm or add other types of constraints to make it more effective for other systems. The assumptions regarding the event mapping specification (see Sect. 3.1) can also be altered. We observed that incorporation of more domain knowledge leads to possibly less generic but more effective model refinement processes.

5 Related work

There exist a large body of work on MBT techniques (Neto et al. 2007), tools (Dalal et al. 1999) and different types of models employed like finite state machines (Robinson 1999; Chander et al. 2011) and Markov chains (Whittaker and Thomason 1994). There also exist surveys on MBT (Neto et al. 2007) and several case studies (Dalal et al. 1999; Keranen and Raty 2011) where the effectiveness of MBT is evaluated. The surveyed studies and the applied MBT techniques are mainly focusing on the modeling approach and test case generation methods. The evaluation is performed by comparing the number of faults detected by MBT with respect to the number of faults exposed by traditional testing approaches. However, iterative refinement of the developed models and the incorporation of domain knowledge have not taken much attention in the literature.

Recent advances in automatic black box testing were summarized by Mariani et al. (2015). In their literature review, existing approaches are grouped into four categories as random testing, MBT, testing with complex inputs, and combinatorial interaction testing. In the area of MBT, it is confirmed that the effectiveness of this approach is limited by the cost of producing high-quality models (Mariani et al. 2015). Hence, automatically inferring these models is considered to be a promising research direction, which has taken the attention of researchers recently. The other alternative is to apply MBT on specified models (Mariani et al. 2015). Inferred models are further grouped into two categories: ripped models and learning-based models (Mariani et al. 2015). Ripped models are automatically extracted from the system and used for MBT. On the other hand, learning-based models are created iteratively in two phases. First, a model is inferred to generate a set of test cases. Second, observations regarding system behavior during the execution of these test cases are used for augmenting the existing model. The process iterates between these two phases. There are several recently proposed approaches (Mariani et al. 2014; Meinke and Sindhu 2013; Nguyen and Memon 2014) that can be considered within this category. Our approach also relies on learning-based models. However, it is distinguished from the existing approaches in two ways. First, these approaches do not exploit ET activities to refine the model. They exploit test cases that are generated from the previous version of the model. Second, model refinement is limited to the augmentation of missing edges and transitions. In our work, we employed Markov chains as the modeling formalism and we also updated probability values associated with transitions.

A detailed literature review regarding specification mining is provided in the Ph.D. thesis of (Zuo 2015). The common approach among the reviewed studies is summarized as the application of data mining or machine learning techniques on source code or execution traces to generate specifications in various formats (Zuo 2015). Collected execution traces and mined specifications are defined at different levels of abstraction. For instance, component interactions are analyzed in Lorenzoli et al. (2008), whereas object behavior models are mined by TAUTOKO (Dallmeier et al. 2012) in the form of finite state automata. In terms of the model abstraction level, our approach is more aligned with the Observe-Model-Exercise* (Nguyen and Memon 2014) approach, which derives a model of the graphical user interface in the form of event-flow graphs. TAUTOKO (Dallmeier et al. 2012) and Observe-Model-Exercise* (Nguyen and Memon 2014) are similar to our approach in the sense that they both employ an iterative model refinement approach. They start with a possibly incomplete model of the application. Test cases are generated and executed while new events or states are identified by monitoring the execution. Then, the model is extended with these events/states. However, these approaches do not exploit ET

activities to refine the model. In addition, the adopted formalisms do not involve transition probabilities to focus test case generation on error-prone execution paths.

MBT and capture-replay testing techniques were previously integrated (Entin et al. 2011). This integration effort focuses on refining and adapting a capture-replay testing tool based on changes in the user interface due to evolution. In our work, we focus on refining test models based on the captured execution traces instead. We did not consider evolution explicitly; however, it is possible to address changes in the system by updating the event mapping specification.

There exists another approach (Yuan and Memon 2010) that proposes the automated refinement of models for testing. The approach employs an event-flow graph (EFG) (Yuan and Memon 2010) to represent test models for testing applications through their graphical user interface (GUI). Cause–effect relationships among the GUI elements are observed at run-time, while the generated test cases are executed. The model is iteratively refined based on the inferred relationships. This approach does not rely on execution traces that are collected during ET activities to refine the model. Instead, the system is executed with a seed test suite that is generated using an existing model of the GUI. The assumption is that this model represents all possible sequences of events that may be executed on the GUI (Yuan and Memon 2010).

There exist other so-called *automated feedback-based techniques* introduced previously to refine the models of SUT or input data for testing (Ferguson and Korel 1996; Michael et al. 2001; Xie and Notkin 2006; Pacheco et al. 2006). Some of these techniques aim at increasing the code coverage (Ferguson and Korel 1996; Michael et al. 2001), whereas some others aim at generating more test cases to increase the coverage of system behaviors (Xie and Notkin 2006; Pacheco et al. 2006). Regardless of their goals, however, all these techniques rely only on the execution of a previously created test suite to obtain feedback. Hence, the feedback that can be obtained is limited by the coverage of the system behavior achieved with this test suite. In our work, we addressed this limitation by coupling MBT and ET processes.

Besides the reported studies on MBT, ET has also been evaluated in the context of industrial case studies (Itkonen and Rautiainen 2005). These studies investigate the impact of human personality (Shoaib et al. 2009), learning styles (Tinkham and Kaner 2003b) and the way that different types of knowledge (Itkonen et al. 2013) are utilized like domain knowledge, system knowledge and general software engineering knowledge. Their conclusions are mainly drawn from interviews that are performed at different companies (Itkonen and Rautiainen 2005).

Exploratory modeling (Robinson 2000) has been introduced as an approach for model development based on the principles of ET. Hereby, a state diagram of the expected behavior of the SUT is developed first. Then, this model is refined by observing the different states and behaviors while interacting with the SUT. However, this approach has no tool support and it was not evaluated in the context of an industrial case study.

In practice, ET is usually performed manually and MBT is applied as a complementary, rather than an integrated approach. To the best of our knowledge, there has been no method supported by a toolset to couple these approaches and utilize the knowledge gained in ET as a feedback for MBT. In this work, we showed that such an approach is effective for detecting faults.

We have previously proposed an approach (Gebizli and Sozer 2014) for refining test models based on the experience and domain knowledge of test engineers. The goal of that approach is to combine MBT with ET to refine SUT models by extending them with missing states and transitions. However, the refinement process in that approach was

manual. ARME automates this process. It also updates the transition probabilities based on the collected usage profiles.

We have recently proposed another approach for the refinement of transition probabilities (Gebizli et al. 2015, 2016). However, the refinement process was manual. In that approach, transition probabilities are modified based on failure risks calculated with static and dynamic analysis. This approach can also be considered as a risk-based testing approach (Felderer and Schieferdecker 2014) since it employs ET to refine test models. Usage profiles that are collected during ET activities help us to focus test case generation for the most error-prone scenarios.

6 Conclusion and future work

We introduced an approach and a toolset for incorporating information from manual exploratory testing activities into model-based testing to increase the effectiveness of models that are used for test case generation. In our approach, the execution traces that are recorded during exploratory testing activities are utilized as feedback for refining test models. We applied our approach in the context of 3 industrial case studies to improve the models for model-based testing of a digital TV system. We could detect real and critical faults. These faults were not detected by model-based testing based on existing models, and they were also missed during the exploratory testing activities.

As a future work, we plan to apply our approach and use ARME for other subject systems such as home appliances products. Currently, ARME is composed of a set of tools⁷ each of which performs a complementary task: one tool determines new states and transitions to be added based on a given execution trace and event mapping specification; another tool takes this information as input and applies the determined extensions on a given model; and a third tool takes the extended model and execution traces as input to calculate probability values for each transition of the model. In the future, we aim at providing a seamless integration of these tools as part of an integrated development environment.

Acknowledgments This work is supported by the joint grant of Vestel Electronics and the Turkish Ministry of Science, Industry and Technology (909.STZ.2015). The contents of this article reflect the ideas and positions of the authors and do not necessarily reflect the ideas or positions of Vestel Electronics and the Turkish Ministry of Science, Industry and Technology. We would like to thank software developers and test engineers at Vestel Electronics for supporting our case study.

References

- Bach, J. (2003). *Exploratory testing explained*, Technical report. <http://www.satisfice.com/articles/et-article.pdf>.
- Belli, F. (2001). Finite state testing and analysis of graphical user interfaces. In *Proceedings of 12th international symposium on software reliability engineering, ISSRE2001* (pp. 34–43).
- Belli, F., Endo, A. T., Linschulte, M., & Simao, A. (2014). A holistic approach to model-based testing of web service compositions. *Software: Practice and Experience*, 44(2), 201–234.
- Briand, L., & Labiche, Y. (2002). A uml-based approach to system testing. *Software and Systems Modeling*, 1(1), 10–42.

⁷ The ARME toolset is available at: <http://srl.ozyegin.edu.tr/projects/armor/>.

- Chander, A., Dhurjati, D., Koushik, S., & Dachuan, Y. (2011). Optimal test input sequence generation for finite state models and pushdown systems. In *Proceedings of the IEEE fourth international conference on software testing, verification and validation* (pp. 140–149).
- Chow, T. (1978). Testing software design modeled by finite-state machines. *IEEE Transactions on Software Engineering*, 4(3), 178–187.
- Dalal, S. R., Jain, A., Karunanithi, N., Leaton, J. M., Lott, C. M., Patton, G. C., & Horowitz, B. M. (1999). Model-based testing in practice. In *Proceedings of the international conference on software engineering* (pp. 285–294).
- Dallmeier, V., Knopp, N., Mallon, C., Fraser, G., Hack, S., & Zeller, A. (2012). Automatically generating test cases for specification mining. *IEEE Transactions on Software Engineering*, 38(2), 243–257.
- Dustin, E., Rashka, J., & Paul, J. (1999). *Automated software testing: Introduction, management, and performance*. Boston, MA: Addison-Wesley Longman Publishing Co., Inc.
- Entin, V., Winder, M., Zhang, B., & Christmann, S. (2011). Combining model-based and capture-replay testing techniques of graphical user interfaces: An industrial approach. In *Proceedings of the 4th IEEE international conference on software testing, verification and validation workshops* (pp. 572–577).
- Felderer, M., & Schieferdecker, I. (2014). A taxonomy of risk-based testing. *International Journal of Software Tools and Technology Transfer*, 16(5), 559–568.
- Ferguson, R., & Korel, B. (1996). The chaining approach for software test data generation. *ACM Transactions on Software Engineering and Methodology*, 5(1), 63–86.
- Gebizli, C. S., & Sozer, H. (2014). Improving models for model-based testing based on exploratory testing. In *Proceedings of the 6th IEEE workshop on software test automation* (pp. 656–661) (**COMPASAC Companion**).
- Gebizli, C.S., Metin, D., & Sozer, H. (2015). Combining model-based and risk-based testing for effective test case generation. In *Proceedings of the 9th workshop on testing: Academic and industrial conference—practice and research techniques* (pp. 1–4) (**ICST Companion**).
- Gebizli, C. S., Sozer, H., & Ercan, A. (2016). Successive refinement of models for model-based testing to increase system test effectiveness. In *Proceedings of the 10th workshop on testing: Academic and industrial conference—practice and research techniques* (pp. 263–268) (**ICST Companion**).
- Gonenc, G. (1970). A method for the design of fault detection experiments. *IEEE Transactions on Computers*, 19(6), 551–0558.
- Guen, H. L., Marie, R., & Thelin, T. (2004). Reliability estimation for statistical usage testing using markov chains. In *Proceedings of the 15th international symposium on software reliability engineering* (pp. 54–65).
- Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3), 231–274.
- Hetzel, W. C., & Hetzel, B. (1991). *The complete guide to software testing* (2nd ed.). New York, NY: Wiley.
- Itkonen, J. (2011). *Empirical studies on exploratory software testing*, Ph.D. thesis. Aalto University.
- Itkonen, J., & Rautiainen, K. (2005). Exploratory testing: A multiple case study. In *Proceedings of international symposium on empirical software engineering* (pp. 84–93).
- Itkonen, J., Mantyla, M. V., & Lassenius, C. (2007). Defect detection efficiency: Test case based vs. exploratory testing. In *First international symposium on empirical software engineering and measurement* (pp. 61–70). IEEE Computer Society.
- Itkonen, J., Mantyla, M. V., & Lassenius, C. (2013). The role of the testers knowledge in exploratory software testing. *IEEE Transactions on Software Engineering*, 39(5), 707–724.
- Joye, C. (2014). *Matelo test case generation algorithms: Explanation on available algorithms for test case generation*. <http://www.all4tec.net/MaTeLo-How-To/understanding-of-test-cases-generation-algorithms.html>.
- Kaner, C. (2006). Exploratory testing. In *Quality assurance institute worldwide annual software testing conference*.
- Keranen, J., & Raty, T. (2011). Model-based testing of embedded systems in hardware in the loop environment. *IET Software*, 6(4), 364–376.
- Lorenzoli, D., Mariani, L., & Pezzè, M. (2008). Automatic generation of software behavioral models. In *Proceedings of the 30th international conference on software engineering, ICSE '08* (pp. 501–510). ACM.
- Mariani, L., Pezzè, M., Riganelli, O., & Santoro, M. (2014). Automatic testing of GUI-based applications. *Software Testing, Verification and Reliability*, 24(5), 341–366.
- Mariani, L., Pezzè, M., & Zuddas, D. (2015). Recent advances in automatic black-box testing. In A. Memon (Ed.), *Advances in computers* (Vol. 99, pp. 157–193). Amsterdam: Elsevier.

- Meinke, K., & Sindhu, M. A. (2013). Lbtest: A learning-based testing tool for reactive systems. In *Sixth IEEE international conference on software testing, verification and validation, ICST 2013* (pp. 447–454). Luxembourg.
- Michael, C., McGraw, G., & Schatz, M. (2001). Generating software test data by evolution. *IEEE Transactions on Software Engineering*, 27(12), 1085–1110.
- Myers, G. J., & Sandler, C. (2004). *The art of software testing*. New York: Wiley.
- Naito, S., & Tsunoyama, M. (1981). Fault detection for sequential machines by transitions tours. In *IEEE fault tolerant computer symposium* (pp. 238–243).
- Neto, A. C. D., Subramanyan, R., Vieira, M., & Travassos, G. H. (2007). A survey on model-based testing approaches: A systematic review. In *Proceedings of the 1st ACM international workshop on empirical assessment of software engineering languages and technologies* (pp. 31–36).
- Nguyen, B., & Memon, A. (2014). An observe-model-exercise* paradigm to test event-driven systems with undetermined input spaces. *IEEE Transactions on Software Engineering*, 40(3), 216–234.
- Pacheco, C., Lahiri, S., Ernst, M., & Ball, T. (2006). Feedback directed random test generation. In *Proceedings of the 29th international conference on software engineering* (pp. 396–405).
- Pretschner, A. (2005). Chap Model-based testing in practice. In *Proceedings of the international symposium of formal methods Europe* (pp. 537–541). Berlin: Springer.
- Robinson, H. (1999). Finite state model-based testing on a shoestring. In *Proceedings of the software testing and analysis and review west conference*.
- Robinson, H. (2000). Intelligent test automation a model-based method for generating tests from a description of an applications behavior. In *Software testing and quality engineering magazine* (pp. 24–32).
- Shoaib, L., Nadeem, A., & Akbar, A. (2009). An empirical evaluation of the influence of human personality on exploratory software testing. In *Multitopic conference and IEEE 13th international* (pp. 1–6).
- Tinkham, A., & Kaner, C. (2003a). Exploring exploratory testing. In *Proceedings of the software testing and analysis and review east conference*.
- Tinkham, A., & Kaner, C. (2003b). Learning styles and exploratory testing. In *Proceedings of the Pacific northwest software quality conference*.
- Tretmans, J. (2011). Chap Model-based testing and some steps towards test-based modelling. In *Formal methods for eternal networked software systems: 11th international school on formal methods for the design of computer, communication and software systems, SFM 2011, Bertinoro. Advanced Lectures* (pp. 297–326). Berlin: Springer.
- Utting, M., & Legeard, B. (2007). *Practical model-based testing: A tools approach*. San Francisco, CA: Morgan Kaufmann Publishers Inc.
- Utting, M., Pretschner, A., & Legeard, B. (2012). A taxonomy of model-based testing approaches. *Software Testing, Verification and Reliability*, 22(5), 297–312.
- Whittaker, J., & Thomason, M. (1994). A Markov chain model for statistical software testing. *IEEE Transactions on Software Engineering*, 20(10), 812–824.
- Whittaker, J. A. (2009). *Exploratory software testing: Tips, tricks, tours, and techniques to guide test design* (1st ed.). Reading, MA: Addison-Wesley Professional.
- Xie, T., & Notkin, D. (2006). Tool-assisted unit-test generation and selection based on operational abstractions. *Automated Software Engineering*, 13(3), 345–371.
- Yuan, X., & Memon, A. (2010). Generating event sequence-based test cases using GUI runtime state feedback. *IEEE Transactions on Software Engineering*, 36(1), 81–95.
- Zuo, Z. (2015). *Refinement techniques in mining software behavior*, Ph.D. thesis. National University of Singapore.



Ceren Şahin Gebizli received her B.Sc. degree in Computer Engineering from Dokuz Eylül University, Turkey, 2010, M.Sc. degree in Computer Engineering from Özyeğin University, Turkey, 2014 and continues on her Ph.D. in Computer Engineering, Özyeğin University, Turkey. Also, she is working as a Test Architect at Vestel Electronics R&D.



Hasan Sözer received his B.Sc. and M.Sc. degrees in Computer Engineering from Bilkent University, Turkey, in 2002 and 2004, respectively. He received his Ph.D. degree in 2009 from the University of Twente, The Netherlands. From 2002 until 2005, he worked as a software engineer at Aselsan Inc. in Turkey. From 2009 until 2011, he worked as a postdoctoral researcher at the University of Twente. He is currently an assistant professor at Özyeğin University.